

Pattern A — Adjacent Component as Input to a CKS Cell: Standalone Treatment of How Adjacent AI Components Inform Cell Reasoning Without Becoming Authoritative for Coordination in CKS

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the three composition patterns named in the source paper's hybrid-systems-composition treatment — **Pattern A: adjacent component as input to a CKS cell** — as a standalone architectural specification, separable from the other two patterns (Pattern B, derived view; Pattern C, separate concern) and from the anti-patterns those positions preempt.

Abstract

The CKS pattern's hybrid-systems-composition treatment names three positions an adjacent AI component can occupy relative to a CKS substrate: as input to a cell (Pattern A), as a derived view (Pattern B), or as a separate concern (Pattern C). A separate note formalizes the integrating frame in which all three are named together. This note formalizes Pattern A as having independent architectural content that can be defended, implemented, and tested independently of the other two. Pattern A is the most operationally common of the three composition patterns: real CKS deployments use it for cells consulting RAG indexes for source-document grounding, fine-tuned LLMs as the LLM-as-mediator inside cell execution, vector databases for semantic search, and external knowledge stores for domain facts. The pattern is operationally consequential because hybrid AI deployments are the dominant 2024–2026 architectural shape into which CKS substrates compose. The note states what Pattern A means precisely, specifies the four operational components of the pattern (substrate as primary source; consultation within cell execution; provenance crosses the component boundary cleanly; cell outputs remain orchestration-rule-governed), distinguishes it from four adjacent variations commonly conflated with it, names the failure modes that violate it, and provides an operational test with three sharpening properties for whether a cell-component boundary satisfies Pattern A's requirements.

1. Why Pattern A needs to be formalized as standalone

The parent foundational note commits CKS to three composition patterns under which adjacent AI components can be positioned relative to the substrate; the integrating-frame note named all three at the

integrating level. The joint framing is correct as far as it goes, and this note does not contradict it. But Pattern A carries the weight of the architecture's interface to the dominant hybrid AI shape of the surrounding period, and reading it only through the integrating frame leaves operational content underspecified at the moment of greatest practical pressure.

Real CKS deployments rarely sit alone, and Pattern A is the position they most commonly use. A regulated coordination workflow with a CKS substrate at its center is typically embedded in a larger AI system: a RAG index over source documents that cells consult for grounding before deciding; a fine-tuned LLM acting as the LLM-as-mediator inside cell execution; a vector database supporting semantic search over related content; an external knowledge store the organization already operates. Each of these is, architecturally, an adjacent AI component being consulted by a cell during the cell's execution. Hybrid AI architectures are the dominant 2024–2026 deployment shape, and the easiest drift away from the CKS architecture happens precisely at this boundary; the integrating-frame note named the three positions, but the per-position operational specification is what makes the pattern testable at the moment a hybrid composition is being designed.

2. Pattern A, defined precisely

In the CKS pattern, **Pattern A** is the architectural position in which an adjacent AI component is consulted by a CKS cell during the cell's execution, and four properties hold throughout the consultation:

1. The cell reads from the substrate as its primary source of truth. The substrate is the primary information source; the adjacent component is additional context.
2. The cell may query the adjacent component for additional context — source-document grounding (RAG index), domain-specialized reasoning (fine-tuned LLM), semantic search (vector database), domain facts (external knowledge store), or other informational purposes.
3. The cell writes its outputs back to the substrate under its orchestration rule. Writes are rule-governed; the adjacent component does not gain substrate-write authority.
4. The adjacent component is part of the cell's reasoning environment; the substrate remains authoritative. The consultation is informational; the cell remains the architectural primary, and the substrate remains the source of truth for coordination questions.

Pattern A is the architectural pattern by which adjacent AI components inform cell reasoning without becoming authoritative for coordination. It does not concern any specific kind of adjacent component (RAG, fine-tuned LLM, vector database, external store); it concerns the architectural relationship between any such component and a cell that consults it.

3. The four operational components of Pattern A

The architectural commitment is specifiable as four operational components. Each must hold for a consultation to instantiate Pattern A; failing any one fails the pattern, even when the other three hold robustly.

(a) Substrate as primary source. The cell reads from the substrate as its primary source of truth. The adjacent component is consulted for additional context but does not replace the substrate as the cell's

primary information source. Where the substrate carries authoritative content for a coordination question, the cell uses the substrate; the adjacent component supplies context the substrate does not carry.

(b) Consultation within cell execution. The adjacent-component consultation occurs within the cell's execution boundary, not outside it. The cell is the architectural unit performing the consultation; adjacent components do not independently initiate substrate interactions, do not run in parallel to cells with their own substrate-write paths, and do not stand alongside cells as peers exercising governance authority. The cell mediates; the adjacent component is consulted by the cell.

(c) Provenance crosses the component boundary cleanly. The cell's writes back to the substrate must be attributable to the cell, the orchestration rule that authorized the write, and the adjacent components consulted. Path retraceability does not stop at the cell boundary; it extends through whatever the cell consulted. A write produced with input from a RAG index, a fine-tuned LLM, or a vector database must record that consultation — index identity and retrieved content references, model identity, query and result-set references, respectively — as substrate provenance. Otherwise the retraceable path breaks at the moment the cell's reasoning crossed an opaque boundary. This component operationalizes the addressable-provenance-across-boundaries composition requirement for consultation-based compositions.

(d) Cell outputs remain orchestration-rule-governed. The adjacent component may inform the cell's reasoning, but the cell's writes are still subject to the rule that authorizes them. An adjacent component does not provide additional write authorization, does not bypass the orchestration rule, and does not extend the cell's authority beyond what the rule defines. The rule may authorize a class of consultations as part of the cell's normal reasoning environment; it does not delegate write authority to whatever the cell happens to consult. The pattern preserves AI-as-substrate-mediator so long as the cell continues to read from and write to the substrate under its rule; the adjacent component is part of the cell's reasoning environment, not a substitute for the substrate or a parallel write path around it.

The four components together define Pattern A architecturally. A consultation that satisfies all four has Pattern A in the architectural sense; a consultation that fails any one has acquired a position the architecture does not name.

4. What Pattern A does NOT claim

The standalone treatment is not a maximalist treatment. Stating precisely what Pattern A does not claim is what keeps the framing from drifting into commitments the source paper does not support.

It does not claim that all hybrid compositions must use Pattern A. Patterns B and C are also legitimate positions; the architectural commitment is that for each adjacent component, the position it occupies is identifiable and the requirements of that position are met.

It does not require all CKS deployments to consult adjacent components. A deployment may have CKS cells that operate without adjacent consultations and remain CKS-coherent. The commitment is that *when* consultations occur, they satisfy Pattern A's requirements.

It does not foreclose substrate-authoritative coordination when consultations disagree. When the substrate and the adjacent component disagree on a coordination question, the substrate wins by definition. The commitment is to substrate authority, not to consultation-based reconciliation.

It does not specify implementation patterns. Implementations may use various mechanisms — API calls, RAG retrieval libraries, vector store queries, fine-tuned LLM invocation, MCP-style tool calls, in-process libraries — and the commitment is to the four operational components, not to any specific implementation.

It does not require every individual consultation to be rule-authorized separately. The orchestration rule may authorize a class of consultations as part of the cell's normal reasoning environment; individual consultations need not each be enumerated as discrete rule clauses.

It does not foreclose cells consulting multiple adjacent components. A cell may consult several adjacent components within a single execution. Each consultation must satisfy Pattern A's requirements, with provenance for all consulted components recorded in the cell's writes.

5. What Pattern A is NOT

Four adjacent variations are commonly conflated with Pattern A. Each is a real shape that hybrid AI deployments take; naming what Pattern A is not is what prevents the misreading.

Not RAG without provenance crossover. Retrieval-augmented architectures where retrieved content informs LLM generation without recording the retrieval as provenance fail the third operational component. RAG systems may produce outputs architecturally similar to Pattern A consultations, but without the provenance trail Pattern A requires the retraceable path breaks at the moment retrieval entered the cell's reasoning. Pattern A admits RAG as a consultation pattern; it does not admit RAG-without-provenance as a Pattern A consultation.

Not fine-tuned-LLM-as-cell-substitute. Architectures where a fine-tuned LLM operates as a substitute for cells — making decisions, writing to the substrate, exercising authority outside the orchestration rule — fail the fourth operational component. Fine-tuned LLMs in Pattern A operate as the LLM-as-mediator inside cells under rules. A fine-tuned LLM that has acquired direct write authority, decision-making authority outside cell scope, or rule-authoring authority has stopped being a Pattern A consultation.

Not external-knowledge-store-as-cell-authority. Architectures where an external knowledge store provides cell-level authority — the store determining what the cell decides, with the cell deferring to the store on coordination questions — fail the first operational component and the source-of-truth commitment. External stores in Pattern A inform cell reasoning; they do not authorize cell decisions.

Not vector-database-as-substrate-substitute. Architectures where a vector database operates as the deployment's substrate — coordination state lives there rather than in the CKS substrate — fail the first operational component. Vector databases in Pattern A are consulted as additional context. A deployment that has migrated coordination state into a vector database has not extended Pattern A; it has migrated out of CKS.

6. Why Pattern A is load-bearing for downstream commitments

Pattern A is load-bearing for several CKS commitments. The integrating hybrid-systems-composition specification depends on it as one of three positions; without it, the architecture has no specification for cell-consultation hybrid compositions. The AI-as-substrate-mediator commitment is preserved across consultations by Pattern A's first, third, and fourth operational components: substrate-as-primary-source preserves the read-substrate-as-primary mediator property; provenance-across-the-boundary preserves the attribution mediator property; orchestration-rule governance preserves the writes-under-rules mediator property.

Path retraceability is extended to cross-component consultations through Pattern A's third component; the six-field provenance architecture applies at the consultation boundary by recording adjacent components consulted as part of the cell write's provenance. The orchestration-rule commitment is preserved by Pattern A's fourth component, which prevents adjacent components from extending or bypassing rule authority. The five source-of-truth commitments — substrate authoritative for what is the case, what is current, what is history, what rules apply, who has what authority — are preserved by Pattern A's first component. The composition-requirements specification, particularly the addressable-provenance-across-boundaries requirement, operates at Pattern A's consultation boundary, where the third component is its operational expression. The hybrid composition coherence specifications elaborated for the three-adjacencies and orchestration-layer notes describe compositions that operate within Pattern A's architectural pattern.

In each case, Pattern A is the pattern that makes the corresponding commitment hold across cell-consultation hybrid compositions. Without it, the commitments would have to be re-derived for each hybrid shape; with it, the architectural relationship between cells and adjacent components is specified once and reused.

7. Failure modes that violate Pattern A

Each of the following is a way an implementation can fail Pattern A by allowing adjacent components to gain substrate-level authority or by losing provenance across the consultation boundary.

(a) Adjacent component writes directly to substrate. The adjacent component writes coordination state to the substrate without cell mediation. Component (b) of §3 fails; the adjacent component has acquired writer authority Pattern A forbids.

(b) Provenance lost at the consultation boundary. The cell consults the adjacent component and writes to the substrate, but the substrate write does not record the consultation. Component (c) fails; the retraceable path breaks at the consultation boundary.

(c) Adjacent component exercises authority beyond the rule. The adjacent component's involvement extends the cell's write capabilities beyond what the orchestration rule defines — the cell writes content the rule would not have authorized had the adjacent component not been consulted. Component (d) fails.

(d) Substrate deferred to adjacent component on coordination questions. The cell defers to the adjacent component when the substrate and the adjacent component disagree on a coordination

question. Component (a) fails; substrate authority is compromised.

(e) Adjacent-component consultation bypasses the orchestration rule. The cell performs consultations the orchestration rule does not authorize as part of its reasoning environment, producing substrate writes that are not rule-governed. Component (d) fails.

(f) Provenance partially recorded. Some adjacent components consulted are recorded but not all — the retraceable path is partial, with gaps where specific consultations were unrecorded. Component (c) fails operationally even when nominally present.

(g) Adjacent component as cell-substitute. The adjacent component becomes the architectural primary; the cell becomes a thin wrapper around it, with the orchestration rule operationally inert because the adjacent component does the actual work. The architectural-primary commitment fails at the consultation boundary.

(h) Implicit consultation without architectural recognition. The cell consults adjacent components operationally but the consultation is not architecturally recognized — the rule does not name it, the provenance does not record it, and Pattern A's requirements are not enforced because the consultation is implicit. The pattern fails by omission.

(i) Authority leakage through repeated consultation. Repeated consultation of the adjacent component for the same coordination question gradually shifts cell behavior away from substrate-authoritative reasoning. The substrate's primary-source status erodes through accumulated consultation effects even when no individual consultation is non-compliant.

(j) Adjacent component as authority-validator. The adjacent component validates cell decisions — the cell decides, the adjacent component approves or rejects, and the substrate write occurs only on approval. The adjacent component has acquired authority the orchestration rule does not assign; component (d) fails.

The ten failure modes share a common shape: the adjacent component has acquired a position the architecture does not name, and the commitments degrade where the position is unnamed. Pattern A's four operational components are what make each failure mode visible at the moment a practical decision is made.

8. Operational test

A consultation instantiates Pattern A if and only if all of the following are true at all times during the consultation's existence:

1. The cell reads from the substrate as its primary source of truth (component (a) of §3).
2. The consultation occurs within the cell's execution boundary (component (b) of §3) — the cell is the architectural unit performing the consultation.
3. The cell's writes back to the substrate carry provenance for the cell, the orchestration rule, and the adjacent components consulted (component (c) of §3).
4. The cell's outputs remain orchestration-rule-governed (component (d) of §3) — the adjacent component informs reasoning but does not authorize writes, bypass rules, or extend authority.

5. Three sharpening properties from the foundational note hold: (5a) every cell write that incorporated adjacent-component input records the adjacent component as provenance; (5b) when the substrate and the adjacent component disagree on a coordination question, the substrate wins; (5c) the orchestration rule defines what the cell may write, and the adjacent component does not extend this.
6. The four operational components hold for every consultation in the deployment, not just nominally but operationally — the architectural commitment is to per-consultation Pattern A satisfaction.

A consultation that fails any of (1)–(6) does not satisfy Pattern A in the architectural sense, even when it operationally appears to use the adjacent component as input to a cell.

9. The one-sentence test

If a cell reads from the substrate as its primary source of truth, consults an adjacent AI component within the cell's execution, records the consultation as provenance in the cell's write back to the substrate, and the write remains governed by the cell's orchestration rule without the adjacent component extending authority, the consultation is Pattern A; if any of these properties is missing, the consultation has acquired a position the architecture does not name.

The one-sentence test names the most operationally distinctive properties — substrate as primary, consultation-within-cell, provenance-crosses-cleanly, rule-governed-output — for any specific consultation. It is operationally useful for analysts and reviewers who need to classify a specific consultation quickly; the four-component specification of §3 plus the three sharpening properties of §8 provide the full architectural definition for cases requiring detailed analysis.

10. Why naming Pattern A as standalone matters

Implementations under pressure to deliver hybrid AI architectures consistently drift toward consultations that do not satisfy Pattern A's requirements. The drift is steady because hybrid AI is operationally attractive — multiple components offer combined capabilities — and because commercial AI products typically combine retrieval and generation without explicit provenance crossover. Audiences understand "the LLM uses RAG" more readily than "the cell consults the adjacent component within its execution boundary, recording the consultation as substrate provenance, with the substrate remaining authoritative." The shorter framing is the easier path; the architectural commitments do not survive it.

Implementations that drift away from Pattern A produce systems where adjacent components acquire positions the architecture does not name. Consequences manifest as substrate-authority erosion, retraceability failures at consultation boundaries, orchestration-rule failures, and architectural-commitment failures across AI-as-substrate-mediator, path retraceability, and the source-of-truth categories.

Naming Pattern A as a standalone architectural commitment — with the four operational components, the limitations, the four adjacent-variation distinctions, the load-bearing connections, the ten failure modes, and the operational tests — gives downstream readers a precise specification of what cell-consultation hybrid compositions must satisfy. Subsequent notes specialize Pattern B and Pattern

C; a further note specializes the anti-patterns and composition coherence. Together they will close the decomposition of the parent foundational note. The integrating frame named the three positions; this note has formalized one of them.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Pattern A — Adjacent Component as Input to a CKS Cell: Standalone Treatment of How Adjacent AI Components Inform Cell Reasoning Without Becoming Authoritative for Coordination in CKS*. May 5, 2026. ORCID: 0009-0004-8065-3235.